

Computer Vision Project – Stereo Robot Navigation

Authors: De Blasi Alessandro, Romeo Simone

Contact: alessandro.deblasi2@studio.unibo.it, simone.romeo3@studio.unibo.it

April 2024

1. INTRODUCTION

1.1. Text of the assignment

Given a video sequence taken by a stereo camera mounted on a moving vehicle, the project's objective is to sense information concerning the space in front of the vehicle which may be deployed by the vehicle navigation system to automatically avoid obstacles.

1.2. Introduction to the codes

We provide two codes, one with optional points("cv_project_optional.py") and one without optional points("main.py").

Our goal is to obtain the best possible depth estimate in both cases.

We will discuss the cv_project_optional.py file, but we will highlight some important differences from main.py.

2. CODE BREAKDOWN

2.1. Initialization

All necessary global variables and empty lists are initialized. Moreover, we capture the stereo videos.

An important parameter to choose was the ROI's dimensions. After some attempts we decided to take a 60x60 square, as it excluded some parts of the background, giving a better estimation of the distance as a result.

2.2. Stereo Vision Setup

We created a simple GUI (some trackbars that are not present in the final code file) to adjust the stereo block matching parameters dynamically ("block_size" and "min_disp").

2.3. Disparity calculation

In the main loop, the script reads frames from both cameras, converts them to grayscale, and computes the disparity map using the StereoSGBM algorithm.

From the disparity in the ROI, we also compute d_{main} which is then used to estimate the distance. We decided to calculate d_{main} as the median of the disparity to ensure that extreme values do not affect the distance estimation.

2.4. Distance Estimation

The disparity values are used to estimate the distance z to the obstacle, using the formulae:

$$z(mm) = \frac{b(mm) \cdot f(pixel)}{d_{main}(pixel)}$$

After a certain threshold (0.8m) is reached, an alert message is shown on video.

2.5. Chessboard measurement

A function **measure_chessboard** is defined to detect a chessboard pattern in the frames and calculate its width and height in pixels, which are then converted to millimeters based on a given distance z .

The corners of the chessboard are then also shown on video.

3. OPTIONAL POINTS

3.1. Dynamic disparity range

We implemented these features by updating **min_disp** and **num_disp** for each frame to achieve a disparity range of 64, with d_{main} always lying in the center of it.

Since these parameters vary with distance, we noticed that the disparity map has less noise than in the “main.py” implementation, as the disparity grows too much at smaller distances from the obstacle.

This implies a smaller error in the estimation of the chessboard.

In the codes we sent, this advantage is not present because the second optional point (ROI divided into stripes) makes the estimation slightly less accurate.

3.2. ROI subdivision into stripes

After several attempts, we decided to divide the ROI into 5 stripes as it gave the best results given the chosen dimension of the ROI (60x60). This operation allows us to sense the depth more accurately since 5 d_{main} are computed. Thus, we measure 5 distances which is optimal (especially when the stereo camera is not parallel to the obstacle) as the minimum distance check is done for each stripe.

In this way, we can estimate the inclination of the stereo camera with respect to the obstacle.

A graphical representation of the distances is also displayed in real time to provide a visual representation of the inclination.

3.3. Some improvements regarding the disparity

We tried to implement a median filter to improve the disparity map, but after some research about the **stereoSBGM** function, we discovered that there were two parameters (P1 and P2) that provided better results than the median filter.

However, since the remaining error on the disparity map, we decided to sort the disparity and exclude the lowest 15% of the values.

Also, to smooth the output of the estimated distance, we implemented a moving average filter.

4. RESULTS

To understand if the code was working as intended, we printed or plotted several values to get a match with real given values (or just to understand if values were plausible).

Our main goal was to optimize the measurement of the chessboard. In the end, we obtained the following mean values across the whole capture (Width x Height):

- $(128.65 \times 187.18)mm$ on “main.py”
- $(129.32 \times 188.22)mm$ on “cv_project_optional.py”

To obtain more accurate measurements, in both cases, we used a median. We also tried to compute a mean after erasing the upper and lower 5% of values (after gathering width and height in two lists and sorting them), but the values were not as good as when the median was used.

As already pointed out, the measurements are slightly better in the “main.py” implementation, as the real measures are $(125 \times 178)mm$.

We also gathered all the values of the distance and plotted them to have a graphical visualization of the motion to check if it was plausible:

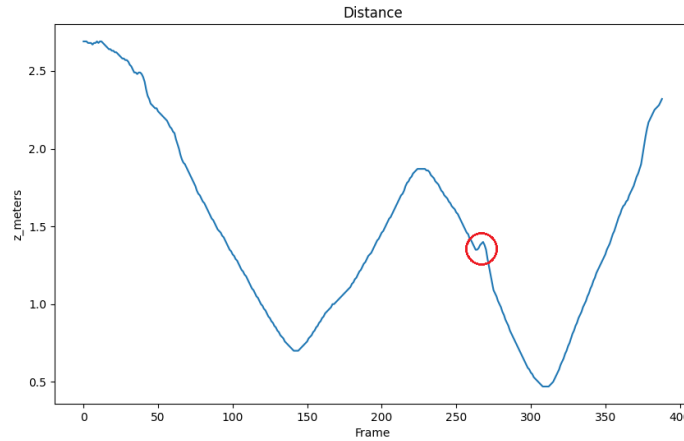


Figure 1 - Plot of z in meters (“main.py”)

In *Figure 1*, the red circle highlights a strange peak in the plot of z for the “main.py”. This is due to the ROI containing a considerable portion of background, which affects the value of d_{main} and, therefore, of the estimated distance. In the implementation of “cv_project_optional.py”, this peak is slightly flattened as an effect of the usage of the subdivided ROI, which makes the “background stripe” weigh less when the mean is computed.

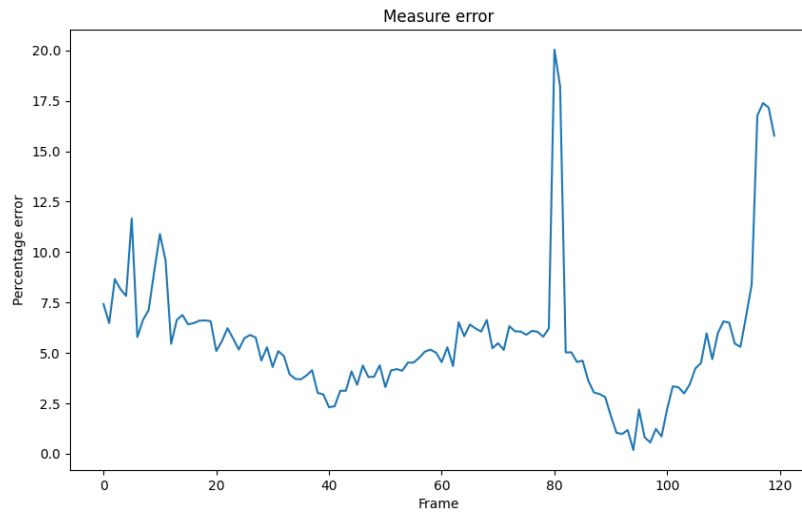


Figure 2 – Percentage error on the chessboard's dimensions ("main.py")

Indeed, we can see a peak in the plot of the error during the second approach maneuver of the robot. Subsequently, the ROI returns on the chessboard and since the distance is short at this point, the error becomes very small. In the final part of the video, we have an analogous situation.

Finally, we also tried to measure the execution time of both codes using the library *time*. However, we found no improvement or degradation between the two implementations.